

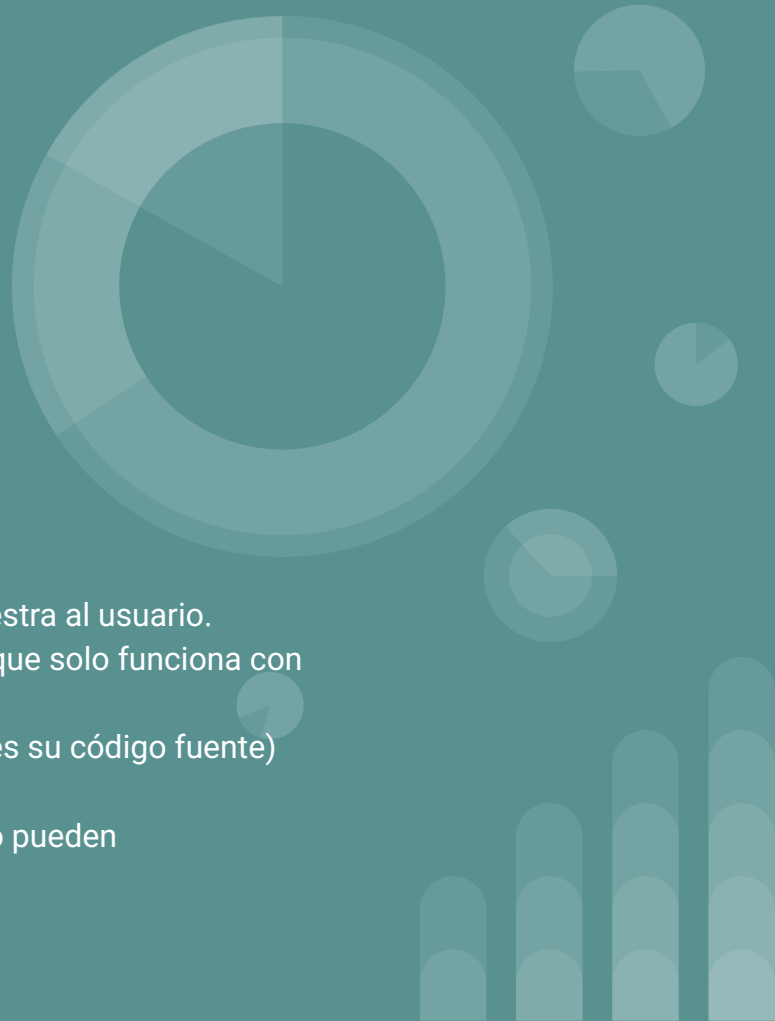
# Patrón de diseño: Adapter (Adaptador)

**Categoría Gof:** Estructural

**Propósito:** Permitir que objetos con interfaces incompatibles colaboren entre sí

**Problema de Diseño:**

- **Contexto:** Tu aplicación descarga datos en formato XML y los muestra al usuario.
- **El Conflicto:** Necesitas integrar una biblioteca de análisis externa que solo funciona con JSON.
- **La Restricción:** No puedes modificar la biblioteca externa (no tienes su código fuente) ni quieres romper tu código actual.
- **Resultado:** Dos componentes con interfaces incompatibles que no pueden comunicarse directamente.



# Idea central (Solución)

El adaptador es un objeto que actúa como traductor entre dos interfaces incompatibles

## Funcionamiento:

1. El código cliente llama a un método del Adaptador (usando la interfaz que ya conoce).
2. El Adaptador traduce la llamada y los datos (ej. XML → JSON).
3. El Adaptador llama al método del servicio externo con el formato que este espera.

El adaptador "envuelve" al objeto incompatible, ocultando la complejidad de la conversión.

## Representación Gráfica



# Ventajas, desventajas y cuando NO usarlo

## Ventaja Clave

- **Principio de Abierto/Cerrado:** Puedes introducir nuevos adaptadores sin modificar el código cliente existente. Esto facilita el mantenimiento y la escalabilidad.

## Desventaja o limitación

- **Aumento de Complejidad:** Introduces nuevas interfaces y clases. A veces es más sencillo modificar la clase de servicio si se tiene acceso a su código.

## ¿Cuándo NO Utilizarlo?

- **Cuando puedes modificar la clase de servicio:** Si tienes acceso al código de la biblioteca incompatible y es más fácil cambiar su interfaz, es más directo hacerlo.
- **Cuando la conversión es trivial:** Si la diferencia entre interfaces es mínima (ej. solo cambia el nombre de un método), un adaptador completo es sobre-ingeniería.
- **Cuando el problema es de lógica, no de compatibilidad:** Si la lógica de negocio es fundamentalmente incompatible, un adaptador no soluciona el problema de fondo.



# Analisis critico de IA, fuentes y demo

## Análisis Crítico de IA:

- Afirmación de IA: "El patrón Adapter se utiliza principalmente para añadir nuevas funcionalidades a un objeto existente sin modificar su estructura."
- Contraste con Fuentes Confiables: Según `refactoring.guru` y el libro "Design Patterns" de la GoF, esto es incorrecto. Añadir responsabilidades a un objeto es el propósito del patrón Decorator.
- Valoración Argumentada: La afirmación es incorrecta porque confunde el propósito de Adapter con el de Decorator. El objetivo de Adapter es la conversión de interfaces para que objetos incompatibles colaboren. Decorator, en cambio, mantiene la misma interfaz pero añade comportamiento. Ambos usan composición, pero sus intenciones son distintas.

## Fuentes:

- Refactoring Guru. (s.f.). *Adapter*. <https://refactoring.guru/es/design-patterns/adapter>
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.

**DEMO en Flutter:** Aplicación Flutter con el patrón Adapter.